Lab 3: Motion Planning with a 6-DOF Manipulator

(72 points total)

1 Main Implementation

In this lab assignment, you will implement the RRT algorithm for a 6-DOF robotic manipulator. To perform the assignment, you will need to have installed the AIKIDO infrastructure. We provide for you the file **adarrt.py**, which contains several methods you will fill in. During development, you will run your code in simulation with -

```
$ python adarrt.py --sim
```

The python file contains the following classes and functions:

- `AdaRRT`: This is the main class. It initializes the start and goal node, the number of iterations, the step_size δ when extending a node in the tree, the desired goal precision ϵ and the joint limits of the robot. It also includes information about the environment, which includes a table, a soda can that we want to grasp, the robot and a set of constraints that check for collisions. This implementation will be very similar to the RRT you built in HW4, with a few modifications discussed below.
- `AdaRRT.Node`: A Node object should contain a copy of the state, a pointer to the parent node in the tree, and a list of pointers to all its child nodes in the tree. A state in the provided code is a 6D `np.array` that contains the robot's configuration.
- `main`: this function specifies the start and goal configurations, sets up the RRT planner and computes a path. It then calls the AIKIDO function `compute_joint_space_path`, which generates a trajectory for the robot to follow.

Steps to complete the lab:

1. **Implement an RRT algorithm** by filling in the code in the provided file. For start-

ing configuration q_S and goal configuration q_G, and parameters ϵ and δ use:

$$q_S = [-1.5, 3.22, 1.23, -2.19, 1.8, 1.2]$$
$$q_G = [-1.72, 4.44, 2.02, -2.04, 2.66, 1.39]$$
$$\delta = 0.25$$
$$\epsilon = 1.0$$

Make sure you have `roscore` running before starting your RRT!

2. **Visualize the trajectory in rviz**. First, execute your `AdaRRT` implementation, but don't execute the trajectory. Then, open rviz from the command line using `rosrun rviz rviz`. In the bottom left module, click the "Add" button and navigate to the "By Topic" tab. You should see a `InteractiveMarkers` topic under `/dart_markers`. Add the topic before executing your trajectory generated by `AdaRRT`.

   (a) Accept the GitHub Classroom invitation
   (GitHub Classroom)

   (b) Download the docker image (we will not use the same docker images as in Lab1 and Lab2).
   If use **ARM-64** architecture:

   - If use Python2:
     Download and directly load this docker image:
     (Google Drive)
     `docker load -i cs545-lab3-melodic-arm64.tar`.
   - If use Python3:
     Download and directly load this docker image:
     (Google Drive)
     `docker load -i cs545-lab3-noetic-arm64.tar`.

   If use **X86-64** architecture:

   - If use Python2:
     Download and directly load this docker image:
     (Google Drive)
     `docker load -i cs545-lab3-melodic-x86-64.tar`.

   If build your own Docker image:

   - Follow this instruction:
     (Google Drive)

   (c) Build the provided workspace (already in the docker image, you can skip this step): (Google Drive)

Unzip it to your home directory. Before running the assignment codes, make sure you have run source /ros_ws/devel/setup.bash in the same terminal as your lab script. Remove build, devel, and log from your ros_ws and rebuild the workspace by running

```
catkin_make_isolated  -j4  # Use 4 parallel jobs
```

(d) Create Docker Container

(e) Run lab3 code

If use **ARM-64** architecture:

- Terminal 1: Run roscore
  Run roscore
  ```
  source /opt/ros/melodic/setup.bash
  roscore
  ```
- Terminal 2: Run your script
  Put your script in ' /ros_ws/src/lab3/adarrt.py'.
  Run your script:
  ```
  source ~/ros_ws/devel_isolated/libada/setup.bash
  export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:/root/ros_ws/src/libada
  python ~/ros_ws/src/lab3/adarrt.py --sim
  ```
- Terminal 3: Run Rviz visualization
  ```
  source /opt/ros/melodic/setup.bash
  source ~/ros_ws/devel_isolated/libada/setup.bash
  export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:/root/ros_ws/src/libada
  rosrun rviz rviz
  ```

If use **X86-64** architecture:

- Terminal 1: Run roscore
  Run roscore
  ```
  source ~/.bashrc
  roscore
  ```
- Terminal 2: Run your script
  Put your script in `~/lab3/adarrt.py`.
  Run your script:
  ```
  source ~/.bashrc
  python ~/lab3/adarrt.py --sim
  ```
- Terminal 3: Run Rviz visualization
  ```
  source ~/.bashrc
  rosrun rviz rviz
  ```

3. **(40 points)** Use an off-shelf screen capture software (e.g., `https://itsfoss.com/kazam-screen-recorder/`) to **record a video** of the trajec-

tory. Include the video in the root of your GitHub repo as a file named `question-3.mp4`.
Answer : Uploaded the video on Github

4. **(10 points)** The RRT trajectory is typically jerky. Typical planners use shortcutting algorithms to make the plath smoother. **Replace the function** `ada.compute_joint_space_path` with `ada.compute_smooth_joint_space_path`. Capture the new trajectory with two videos – one showing the default isometric view, and another showing the top view. Include the videos in the root of your GitHub repo as files named `question-4-default.mp4` and `question-4-top.mp4`.

   Answer : Uploaded the video on Github

5. **(10 points)** The goal precision ϵ of 1.0 in the previous question is too large. In order to avoid collisions, we need to improve the precision. However, this dramatically increases the time to compute a solution. To improve computation, **add a method** `_get_random_sample_near_goal` that generates a sample around the goal within a distance of 0.05 along each axis of the search space. Then, change the `build` method so that it calls `_get_random_sample_near_goal` with probability 0.2 and `_get_random_sample` with probability 0.8. Reduce ϵ to 0.2.

   Write down your observations in the PDF file. Also capture the new trajectory with two videos – one showing the default isometric view, and another showing the top view. Include the videos in the root of your GitHub repo as files named `question-5-default.mp4` and `question-5-top.mp4`.

   Answer : Implementing goal-biased sampling in the Rapidly-Exploring Random Tree (RRT) algorithm, combined with reducing the goal precision (ϵ) from 1.0 to 0.2, significantly improved the path quality without causing a prohibitive increase in computation time. The reduced ϵ resulted in a much more precise path, evidenced by the second-to-last waypoint being only 0.09 units from the goal, compared to 0.52 units previously. This tighter requirement also led to a more refined path with more waypoints (11 versus 8). Crucially, the 20% goal-biased sampling strategy proved highly effective, keeping the computation time minimal (only a small increase), successfully allowing the RRT to converge quickly on the restricted goal region, a task that would have been far slower with purely uniform sampling.

6. **(10 points)** Explain why it is not a good idea to call `_get_random_sample_near_goal` with probability 1.0. Also present an example where this could be problematic. Write your answer in the PDF.

   Answer:

1.1 Principle of Exploration vs. Exploitation

The Rapidly-Exploring Random Tree (RRT) algorithm relies on a critical balance between two phases:

- **Exploration:** Uniformly sampling the entire configuration space (C-space) to discover new regions and grow the tree outward from the start configuration $\mathbf{q}_{\text{start}}$.
- **Exploitation:** Biasing samples toward the goal $\mathbf{q}_{\text{goal}}$ to efficiently converge when the tree is already near the target.

A non-zero probability of uniform sampling is essential for the RRT to navigate complex or constrained environments.

1.2 The Failure of $P(\textbf{Sample Near Goal}) = 1.0$

Setting the probability of sampling near the goal to 1.0 completely removes the exploration phase and leads to algorithmic failure.

Consequences

(a) **Failure to Connect:** If every random sample lies in a tiny region around $\mathbf{q}_{\text{goal}}$, the tree repeatedly attempts to extend from $\mathbf{q}_{\text{start}}$ toward the distant goal region. This often fails due to:
   - large distance between start and goal,
   - obstacles blocking direct paths,
   - extension step size limits.

   The tree never grows into unexplored areas of the C-space and remains stuck near the start.

(b) **Loss of Probabilistic Completeness:** RRT is probabilistically complete only if all regions of free space have a non-zero probability of being sampled. Restricting samples to a tiny region near the goal eliminates this property; the algorithm can succeed only if the start already lies within that sampling region.

1.3 Problematic Example: Obstacle-Separated Start and Goal

Consider a workspace where the start and goal configurations lie on opposite sides of a large obstacle.

- **Required Path:** The valid path requires the RRT to explore a long, winding route around the obstacle.
- **Failure Mode When $P = 1.0$:**
  - All $\mathbf{q}_{\text{rand}}$ samples are located near the goal, on the far side of the obstacle.
  - The tree repeatedly attempts extensions from the start directly toward these samples.
  - Each attempted extension intersects the obstacle, causing collision-check failures.

Because there is no uniform sampling, the tree never explores the detour path around the obstacle. A nonzero uniform sampling probability is required to let the tree expand into free space, navigate around obstacles, and eventually reach the goal region.

In-person lab: Once you are confident in your simulation results, you are ready to run it on the real robot. This can be done on the lab workstations with -

```
$ python adarrt.py --real
```

Refer to Piazza for more instructions on scheduling time in the lab.

2 Additional Questions

As a very rough guideline, we anticipate that for most teams, the answers to each question below will be approximately one paragraph long (or about 4-5 bullet points). However, your answers may be shorter or longer if you believe it is necessary.

2.1 Resources Consulted

Question: **(1 points)** Please describe which resources you used while working on the assignment. You do not need to cite anything directly part of the class (e.g., a lecture, the CSCI 545 course staff, or the readings from a particular lecture). Some examples of things that could be applicable to cite here are: (1) did you get help from a classmate *not* part of your lab team; (2) did you use resources like Wikipedia, StackExchange, or Google Bard in any capacity; (3) did you use someone's code (again, for someone *not* part of your lab team)? When you write your answers, explain not only the resources you used but HOW you used them. If you believe your team did not use anything worth citing, *you must still state that in your answer* to get full credit.
Answer : We have used Google Search and github repos for reference to verify `numpy` syntax and the concept of RRT exploration vs. exploitation. No code from classmates or external code bases was used. We have also used the Gemini model to understand the implementation logic for the goal-biased RRT functions.

2.2 Team Contributions

Question: (1 points) Please describe below the contributions for each team member to the overall lab. *Furthermore, state a (rough) percentage contribution for each member*. For example, in a team of 4, did each team member contribute roughly 25% to the overall effort for the project?

- **Shaunak 20%:** Contributed to implementing core RRT functions, including node expansion logic and collision-checking integration. Helped debug joint-space sampling issues and validated simulation results in RViz.
- **Parth 20%:** Focused on implementing goal-biased sampling and tuning parameters such as δ and ϵ. Conducted performance testing to measure the impact of goal-biased sampling on convergence and precision.
- **Shubham 20%:** Implemented the trajectory generation pipeline and executed the required experiments for the default, top-view, and smoothed trajectories. Ensured consistent ROS environment setup and ran multiple trials to compare path quality.
- **Prachi 20%:** Analyzed results for each experiment, interpreted improvements from smoothing and goal biasing, and verified the correctness of RRT behavior across all test cases. Helped ensure that videos and simulations reflected accurate algorithm performance.
- **Hitanshu 20%:** Led troubleshooting efforts for environment setup, including Docker configuration, ROS workspace builds, and integration with AIKIDO. Assisted with validating that the RRT planner handled obstacles correctly across different scenarios.